

Projects	Group Database Projects
IT214 Database Management System, Autumn'2026; pm_jat @ daiict	

To get hands-on, it is expected that you design and implement a database system for a relatively large and real-life situation.

For the project, you will work in groups, and the permitted team size is 4 to 5. You are required to form the group, however, you should be forming the group within your respective lab group only.

Project Evaluation Parameters:

1. Size and Complexity of Schema.
2. Realistic to the real world
3. Complexity of queries answered
4. Goodness of the solutions

Project Milestones -

1. **Write a description of the scenario.** [Due Date: **10-Mar-2026**]

There is no standard format for scenario description, the idea is you must be able to state the scope of the database and capture all data requirements from the description. A sample scenario is being provided for your reference.

Important Note: To identify the scope of the database project; try to understand the database from the user's perspective what purpose this database will serve to the user, what questions of the user, the database will be able to answer, and so forth. Do not think in terms of tables or entities.

2. Draw an **ER Diagram** for the scenario [Due Date: **14-Mar-2026**].

ER diagram should include Cardinality and participation constraints. Use Ternary Relationships, and generalization/specializations judiciously; use them only when you do not find any other way out.

Use the software Dia for creating ERDs. The software is in the software folder in my lecture folder.

3. Create the **relational schema diagram** from your ER diagram. Identify all Functional Dependencies in your Database and prove that all relations, you have created are in BCNF. If not in BCNF decompose it to bring in BCNF, if you cannot, then give a reason, why you can't.

Implement the database as a separate schema on the PostgreSQL server. Save DDL scripts for creating schema.

Submit: (1) Normalized Relational Schema Diagram (2) Minimal FD set (3) Normalization Proofs (4) DDL scripts of your project database.
[Due Date: **28-Mar-2026**]

4. **Retrieval queries on your project database.** You should include as many queries as your scenario may demand. I expect each scenario should have about 10-15 GOOD queries. Test your queries on sample database (Populate your database with sample data through INSERT statements and save INSERT statements in a script file) [Due Date: **4-Apr-2026**]
5. **Stored procedures and Triggers** in your project scenario. Identify some functionality and constraints that can be better accomplished by stored procedures and triggers. [Due Date: **11-Apr-2026**]
6. Create a **simple console-based program** (No GUI required, a short description is given in Annexure-A at the end of this document) using either “embedded SQL in C” or “JDBC in Java” that accesses your project database [Due Date: **11-Apr-2026**].
7. Final document containing ERD and relational schema with normalization proofs. SQL statements of all the queries including DDL INSERT queries. Also submit code of your stored procedures and console application. [Due Date: **18-Apr-2026**]
8. Final Presentation and Viva [**20-Apr -2026 to 24-Apr-2026 on respective lab day**]:
For final viva and presentations, you are to bring hard copies as follows –
 - a. ERD (1 copies)
 - b. Relational Schema Diagram (2 copies)
 - c. Note: Relational schema should be at least in 3NF; BCNF desirable.
 - d. Set of FDs (no MVDs) normalization proofs. (1 copy).
 - e. If any relation is not in BCNF, give a reason why it could not be decomposed into BCNF
 - f. SQL Scripts of SELECT queries (1 copy)
 - g. Source Code of your stored procedures and console application. (1 copy)

Annexure-A

Instructions for console application in C/Java-

Create following functions.

```
void ExecuteQuery(SQL_String) {  
    //Executes SELECT type of sql statement and displays  
    //returned rows one row on a line on "stdout" if no error,  
    //otherwise displays error details */  
}  
  
void ExecuteUpdate(SQL_String) {  
    //Executes INSERT/UPDATE/DELETE type of sql statement  
    //and displays "successful" on stdout if no error,  
    //otherwise displays error details.  
}  
  
void main() {  
    /*  
    Does following tasks:  
    1. test ExecuteQuery and ExecuteUpdate for few of your  
    sample queries.  
    2. Add couple of rows in one of the table using  
    "Prepared Statement"  
    */  
}
```